

Advanced Architecture and Implementation Patterns for Agentic AI Systems: A Comprehensive Snippet Library

Introduction

The transition from stateless large language model (LLM) text generators to autonomous, stateful agentic systems represents a profound paradigm shift in artificial intelligence architecture. Modern foundation models have evolved to natively support multi-step tool execution, explicit reasoning phases, and complex agentic loops. This document serves as an exhaustive architectural reference and code snippet library detailing the implementation of agentic features across six frontier model families: Anthropic's Claude, Google's Gemini, Google's Gemma, Meta's Llama, Microsoft's Phi, and Qwen/DeepSeek.

By analyzing cutting-edge implementations, inference platform documentation, and software development kits (SDKs), this report abstracts the operational mechanics of these models across ten distinct categories. These categories encompass prompt layers, skills and procedures, tools and actions, memory systems, reasoning and planning, agent loops, context management, security and safety, evaluation and testing, and deployment operations. Furthermore, the analysis outlines the necessary security structures and compliance protocols required for deploying Model Context Protocol (MCP) servers in highly regulated enterprise environments. The report concludes by distilling these features into a definitive design template for modern MCP-based agentic architectures.

Prompt Layers: The Contextual Foundation

The prompt layer architecture dictates the model's persona, environmental awareness, and operational boundaries. Modern systems differentiate heavily between system-level directives (which carry operator-level authority) and user-level inputs (which provide localized instructions).

Across the different model families, the injection of this context requires distinct token formatting and API structures. The Claude family utilizes a top-level system parameter within its Messages API, effectively separating the developer's operational guardrails from the user's conversational turns¹. Recent architectural advancements in Claude (specifically Opus 4.5 and 4.6) demonstrate an exceptionally high responsiveness to system prompts, rendering legacy aggressive prompt engineering (e.g., using all-caps directives like "CRITICAL: You MUST") unnecessary and potentially counterproductive, as it may lead to tool over-triggering². Meta's Llama 3.1 relies on explicit header IDs to delineate roles. The system prompt is constructed using specific token boundaries, notably introducing an ipython role prompt that

natively triggers code interpretation behaviors³. Microsoft's Phi-4 requires a ChatML-inspired format, allowing developers to inject JSON-stringified tool schemas directly into the system or developer message⁵. The default Phi-4 template natively expects a system message, though developers can bypass this by passing specific disable parameters to the tokenizer⁶. Similarly, the Qwen 2.5 and DeepSeek families maintain compatibility with ChatML formatting and rely heavily on this layer for robust role-play implementation, condition-setting, and meta-prompting⁷. Google's Gemma 2, specifically the FunctionGemma variant, utilizes a specialized developer role combined with explicit control tokens. For Gemma, the system prompt acts as a mechanical trigger, requiring a highly specific string to activate function-calling capabilities⁹.

Model Family	System/Developer Tag Structure	User/Assistant Tag Structure	Unique Role / Meta-Prompt Features
Claude 3.5/3.7	API system parameter; Mid-conversation {"role": "system"}	{"role": "user"}, {"role": "assistant"}	Prefilled assistant prompts to force output structure; Mid-conversation system injections.
Llama 3.1	< start_header_id >system< end_header_id >	< start_header_id >user< end_header_id >	Environment: ipython role to trigger code execution loops and < eom_id >.
Phi-4	< im_start >system< im_sep >	< im_start >user< im_sep >	System instructions heavily encode {Thought section} reasoning requirements.
Qwen 2.5 / DeepSeek	< im_start >system\n	< im_start >user\n	High resilience to diverse system prompts for complex persona role-play.
Gemma 2	<start_of_turn>developer	<start_of_turn>model	Requires exact trigger phrase in

			developer prompt to unlock tools.
Gemini 2.5/3.0	system_instruction API parameter	input parameter in Interactions API	Native stateful and stateless modes dictating history handling.

Skills & Procedures: Codifying Operational Logic

Moving beyond basic personas, agentic models require codified skills and procedures to execute complex workflows. This category encompasses the heuristics, checklists, policies, and templates that guide the model's decision-making process.

In the Claude ecosystem, skills are codified through structural prompt templates. Best practices dictate using XML tags to create clear hierarchies for checklists and policies². Instead of using negative constraints (e.g., "do not use markdown"), developers achieve higher compliance by establishing positive heuristics and templates (e.g., "Your response should be composed of smoothly flowing prose paragraphs")². Furthermore, for coding agents, documentation generation skills are explicitly mapped using system prompts designed to analyze codebases and produce standardized templates, such as CLAUDE.md documentation files¹¹.

Llama 3.1 codifies skills through strict output formatting templates injected into the system prompt. For instance, when acting as an evaluator or judge, the model is provided with a heuristic checklist (e.g., "Compare the factual content... Ignore differences in style") and a strict JSON template to ensure the resulting output aligns with programmatic parsers¹². The Qwen 2.5 and DeepSeek models leverage their expansive 128K context windows to absorb massive policy documents, allowing developers to inject comprehensive organizational heuristics directly into the system layer⁷.

Tools & Actions: The Execution Layer

The transition from text generation to environmental interaction relies entirely on the tools and actions layer. This encompasses the schemas, API tools, code execution environments, and native browser interactions that allow models to affect the external world.

Tool Schemas and API Integration

The mechanism for defining and invoking tools varies significantly across models. The Claude family defines tools using a JSON Schema-compatible `input_schema`. When the model decides to invoke a tool, it halts generation and returns a `stop_reason` of `tool_use`, at which point the host application executes the tool and returns a `tool_result`¹³. The Anthropic API constructs a specialized internal system prompt containing these definitions, ensuring the model

understands its available action space¹⁵.

Conversely, Google's FunctionGemma eschews standard JSON parsing in favor of an intricate string delimiter mechanism. To prevent JSON parsing errors associated with nested quotes, Gemma utilizes a single special token, <escape>, wrapping string values within the function call payload⁹. This ensures that special characters are treated literally during the API tool invocation. Phi-4 incorporates function calling natively, requiring tools to be injected as JSON dump structures directly into the system prompt wrapped in <|tool|> and </tool|> tokens⁵. Upon tool selection, it outputs a raw JSON object containing the function name and arguments, which the inference engine must catch, execute, and append back into the message history as a tool response¹⁷.

Code Execution and Browser Tools

Meta's Llama 3.1 features a native code execution tool layer. When the system prompt includes the Environment: ipython directive, Llama 3.1 triggers an internal code interpreter mode. The model wraps Python code in <|python_tag|> and terminates it with an <|eom_id|> (End of Message) tag⁴. This specific token acts as a multi-step reasoning indicator, signaling to the execution framework that the model expects a continuation message containing the raw stdout or stderr output of the tool call before proceeding⁴.

Anthropic provides unparalleled browser and desktop interaction capabilities through its computer_20251124 tool. This feature grants the Claude agent the ability to execute screen captures, coordinate-based mouse clicks, and keyboard inputs, effectively allowing it to drive a virtual desktop environment¹⁴. The integration requires the definition of display dimensions within the tool schema and necessitates a recursive agent loop to process the graphical tool results¹⁴.

Model	Schema Definition Strategy	Code Execution Mechanism	Core Interaction Tokens
Claude	JSON Schema in tools array	computer_20251124, bash tools via Agent SDK	tool_use, tool_result
Llama 3.1	Custom system prompt JSON formatting	ipython environment directive	< python_tag >, < eom_id >
FunctionGemma	Developer prompt <start_function_declaration>	External API routing	<escape>, <start_function_call>
Phi-4	Stringified JSON	External application	Raw JSON outputs

	array in < tool > tags	routing	matching schema
--	------------------------	---------	-----------------

Memory Systems: State Persistence and Retrieval

Agentic systems operating over extended time horizons generate vast quantities of conversational state, necessitating advanced memory management. The architecture of memory is bifurcated into localized working memory and externalized long-term, semantic, and episodic memory systems.

Working memory is strictly bounded by the model's context window. Claude, Llama 3.1, and Phi-4 support context lengths of up to 128,000 tokens, while Qwen 2.5 and Gemini can process significantly more⁷. To manage working memory efficiently, Claude implements aggressive prompt caching, allowing the API to read stable conversation history directly from cache, drastically reducing the latency and cost associated with recurrent working memory²¹.

As the agent loop expands beyond the context limit, Retrieval-Augmented Memory (RAG) and semantic memory systems become mandatory. While not natively embedded within the models themselves, Enterprise Agent Platforms utilize Vector Databases to externalize state. Instead of loading massive documents into the context window, an agent invokes an API tool to semantically search a vector store, extracting highly relevant text chunks. This episodic memory formulation—where an agent queries its own past actions and insights—is crucial for multi-session user modeling and long-term task continuity.

Reasoning & Planning: Deliberate Mode and Thinking Configurations

The most significant architectural advancement in current-generation models is the formalization of reasoning and planning frameworks—such as ReAct, Plan-and-Execute, and Reflexion—into explicit, native "thinking" pathways. Instead of relying solely on implicit next-token prediction, models now support Deliberate Mode, generating hidden or explicit reasoning chains before committing to a final action.

Anthropic Claude: Extended and Adaptive Thinking

Claude 3.7 Sonnet introduces a hybrid architecture that seamlessly transitions between standard generation and extended thinking modes. Operators can allocate an explicit "thinking budget," forcing the model into a deliberate planning state¹⁹. The reasoning occurs via an interleaved thinking process, exposing internal self-correction, Tree-of-Thought exploration, and self-critique steps that dramatically enhance performance on complex logic and coding tasks¹⁹.

The API implementation requires defining a thinking block with type: "enabled" and a target budget_tokens²⁴. The budget must meet a minimum threshold of 1,024 tokens and must be strictly less than the max_tokens limit²⁴.

The financial and architectural implications of this configuration are profound. Because thinking

tokens are billed at the premium output rate, unbounded reasoning loops present a severe financial risk for simple queries²¹. Furthermore, changing the budget_tokens between API calls mathematically alters the prompt prefix, instantly invalidating prompt cache breakpoints²⁴. On newer platforms (such as Opus 4.6 and Sonnet 4.6), thinking has transitioned to an adaptive mode, dynamically adjusting effort levels without requiring explicit integer budgets, mitigating runaway token burn²⁵.

Google Gemini: Thought Signatures and Interactions

Gemini 2.5 and 3.0 models implement an internal reasoning process accessible via the ThinkingConfig object within the generative configuration²⁷. Instead of injecting text into the standard output stream, Gemini utilizes the Interactions API to surface reasoning as dedicated, chronologically sequenced "thought steps" alongside function calls and user inputs²⁸. Each thought step contains a mandatory signature—an encrypted representation of the model's internal reasoning state—and an optional summary²⁸. This separation allows the model to perform highly complex multi-step planning while keeping the raw logical execution hidden from the end user unless explicitly requested via the thinking_summaries parameter²⁸.

Phi-4 and DeepSeek: Structured Output Reasoning

Microsoft's Phi-4 utilizes a prompt-enforced reasoning structure rather than native API parameterization. The system explicitly instructs the model to separate its output into two distinct blocks: a {Thought section} and a {Solution section}⁶. This enforces chain-of-thought, self-critique, and reflection prior to final synthesis, allowing developers to parse out the intermediate logic⁶. Similarly, DeepSeek R1 models execute reasoning protocols through an explicit think content block exposed via the API, though integration with platforms like SillyTavern requires specific configuration to render the reasoning content visible²⁹.

Agent Loops & Execution Models

The orchestration of prompts, tools, and reasoning capabilities requires robust Agent Loops. These execution models transform singular request-response interactions into autonomous, task-oriented systems capable of multi-agent delegation.

Autonomous Loops and Agent Frameworks

The Anthropic claude-agent-sdk abstracts the recursive tool-call loop entirely, representing a pinnacle in autonomous loop design. Instead of developers manually handling the cyclical tool_use and tool_result event triggers, the SDK's query() asynchronous iterator automatically manages code execution, file operations, and terminal commands³⁰. The SDK natively handles the context window, seamlessly passing tool results back to the Claude model until the task achieves its terminal state¹⁴.

For Llama 3.1 and Phi-4, execution models are heavily customized by developers using task-oriented agent frameworks like Composio or LangChain. The loop continuously passes the chat history back into the completion engine. In the case of Llama 3.1, the loop explicitly looks for the <|eom_id|> token; when detected, the framework pauses the model, executes the

associated Python script or API call, appends the result to the messages array, and resumes inference⁴.

Multi-agent systems and delegation models emerge when one agent's tool schema is simply the invocation API of another specialized agent. For instance, a generalized planning agent might output a tool call directed at a specialized coding agent, simulating a hierarchical delegation model entirely within the standard function-calling paradigm.

Context Management: Compression and State Persistence

Agents generate immense dialogue histories during task execution. Context management frameworks focus on compression, summarization, chunking, and state persistence to maintain operational efficiency.

A critical challenge in long-running autonomous loops is modifying the agent's operational parameters—such as a session deadline or a tool-availability change—without reprocessing the entire 100K+ token conversation history. Anthropic introduced mid-conversation system messages to address this state persistence challenge²².

By appending a {"role": "system"} message dynamically into the messages array, developers can inject authoritative, operator-level facts mid-loop. Because prompt caching reads prefixes sequentially, injecting a system message *after* a cached breakpoint preserves the cache integrity of previous turns. This technique prevents the invalidation of the massive, stable prefix, drastically reducing latency and token costs while forcing the model to acknowledge a sudden change in state or permissions²².

For data-heavy ingestion, chunking and windowing strategies are mandated. Best practices indicate placing longform data at the absolute top of the prompt window to optimize the attention mechanism's performance across all models².

Security & Safety: Sandboxing and Permission Models

The integration of agentic features, particularly code execution and browser interactions, introduces profound security vulnerabilities. The Model Context Protocol (MCP) bridges foundation models with local and enterprise data environments, demanding rigorous security and safety architectures.

Agentic tools are inherently vulnerable to prompt injection attacks. Malicious instructions hidden on a fetched webpage or within an analyzed file could override the agent's system prompt, hijacking the autonomous loop¹⁴. Anthropic's computer use integration explicitly warns of these risks, mandating that the environment be sandboxed within a dedicated virtual machine or container with minimal privileges¹⁴.

Algorithmic defenses require the deployment of dedicated, lightweight classifier models running asynchronously to evaluate user inputs and tool responses for injection signatures before they reach the primary reasoning model¹⁴. At the operational layer, frameworks like the Claude Agent SDK rely on explicit permission models. The `allowed_tools` configuration creates a strict allowlist; any tool invoked outside this list gracefully falls back to a Human-in-the-Loop

(HITL) `permission_mode`, forcing cryptographic or manual approval before executing potentially destructive actions like file edits³⁰.

Evaluation & Testing: Benchmarking Autonomous Behavior

The non-deterministic nature of reasoning models necessitates advanced evaluation and testing frameworks. Standard unit tests are insufficient for measuring an agent's ability to plan, self-correct, and utilize tools over long horizons.

Evaluation relies on simulated environments and LLM-as-a-judge methodologies. For example, Llama 3.1 is frequently evaluated using frameworks like Braintrust, where the model acts as an evaluator making function calls. The benchmark passes a Factuality YAML template into the model, asking it to compare the factual content of a submitted answer against an expert answer¹². The model uses a predefined tool schema, `select_choice`, to output its reasoning and final score strictly in JSON¹².

Coding agents are tested through automated unit tests dynamically generated by the model itself. Llama 3.1 can be prompted to write comprehensive unit test suites enclosing assertions within specific `[TESTS]` tags, which a continuous integration framework can extract and run in an ephemeral sandbox to benchmark the model's logical accuracy³⁴.

Deployment & Operations: Infrastructure and Observability

Deploying multi-agent systems shifts the engineering challenge from model architecture to cloud infrastructure, observability, and cost optimization.

Hosting open-weight models like Llama 3.1, Phi-4, and Qwen 2.5 requires robust inference platforms such as vLLM or SGLang to manage continuous batching and PagedAttention efficiently⁶. Docker Agent integrations abstract these underlying engines, allowing developers to route commands seamlessly across models using unified configuration files²⁵.

Cost optimization is intricately tied to token usage and observability. As noted with Claude 3.7, the integration of extended thinking mechanisms generates hidden outputs. Observability platforms like Langfuse are employed to capture exact token burns across agent steps, monitoring these reasoning traces that might otherwise bypass standard logging³⁵. For enterprise billing, Anthropic utilizes Claude Consumption Units (CCU) metered directly to AWS or Azure marketplaces, converting raw token usage into a standardized financial metric²¹.

Platform / Tool	Operational Focus	Integration Capability
vLLM / SGLang	High-throughput Inference	Native serving for Phi-4, Llama 3.1, Qwen 2.5 ⁶ .

Docker Model Runner	Agent Gateways	Auto-adjusts max tokens and handles adaptive thinking ²⁵ .
Langfuse	Observability	Captures thinking_budget and thought steps across API calls ³⁵ .
AWS / Azure Bedrock	Enterprise Platforms	Standardized CCU billing, Zero Data Retention (ZDR) routing ²¹ .

Security Standards and Compliance Protocols

Because MCP servers expose local filesystems, APIs, and databases directly to autonomous agents, their design must be constrained by rigorous regulatory compliance protocols. The following structures are required for enterprise deployment, segmented by regulating agency and framework.

General Data Protection Regulation (GDPR) Compliance

GDPR mandates data minimization, purpose limitation, and the right to be forgotten. MCP architectures inherently risk ingesting Personally Identifiable Information (PII) into an agent's working memory or externalized semantic logs.

- **Data Minimization Structures:** MCP servers must enforce strict scope constraints. When an agent requests context from a database tool, the server must utilize context chunking to return only the specific, anonymized paragraphs required, rather than returning entire, unredacted user records.
- **Zero Data Retention (ZDR):** The API pipeline must route through ZDR enterprise endpoints (such as specific configurations in AWS Bedrock), ensuring that user data processed during inference is not persistently logged by the foundation model provider for future model training²⁴.

Health Insurance Portability and Accountability Act (HIPAA)

Handling Protected Health Information (PHI) via an agent requires end-to-end encryption and strict identity verification to maintain compliance with federal healthcare regulations.

- **Credential Handling:** MCP designs must utilize OAuth2 or mutual TLS (mTLS) for agent gateways, completely abstracting underlying database credentials from the LLM. The agent should only ever be provisioned with short-lived session tokens.
- **PHI Masking:** A mandatory middleware interception layer must detect and mask PHI elements (e.g., patient names, social security numbers) prior to the data entering the LLM's context window. These elements must be substituted with cryptographic hashes that the middleware maps back to plaintext securely post-inference.

Service Organization Control 2 (SOC 2)

SOC 2 compliance hinges on security, availability, and processing integrity. In an agentic context, this demands absolute auditability of the autonomous loop.

- **Observability Logging:** Every state transition within the agent loop—including the internal reasoning trace, tool schema invocation, and the exact tool payload—must be immutably logged to an external observability platform³⁵.
- **Human-in-the-Loop (HITL) Checkpoints:** High-risk tool schemas must inherently trigger a HITL breakpoint, halting the execution model until an authorized human entity signs a cryptographic approval token, satisfying SOC 2 processing integrity requirements¹⁴.

Conclusion: The Modern MCP Design Template

By analyzing the exhaustive index of agentic features across the Claude, Gemini, Gemma, Llama, Phi, and Qwen model families, clear abstractions of function by purpose emerge. Distilling these groups of features yields a definitive design template for building modern, compliant Model Context Protocol (MCP) servers.

1. **The Intent Abstraction (Prompt & Reasoning Layer):** Modern MCPs must decouple the system prompt from the reasoning state. The design utilizes a primary **System Interface** relying on dynamic mid-conversation message injection to update operational state without destroying prompt caches²². Reasoning is isolated into a **Deliberate Thinking Pipeline**, utilizing configurable budgets (e.g., Anthropic's `budget_tokens` or Gemini's `ThinkingConfig`) to enforce pre-action validation and self-critique prior to executing any tool²⁴.
2. **The Interface Abstraction (Tools & Context Layer):** The server design standardizes all environmental interactions through strict JSON Schema contracts. It utilizes a **Tool Gateway** that safely handles format idiosyncrasies—such as FunctionGemma's `<escape>` wrappers or Llama 3.1's `<|python_tag|>` outputs—translating them into uniform programmatic actions⁴. A parallel **Context Manager** automates prompt caching and semantic chunking to maintain working memory efficiency.
3. **The Execution Abstraction (Loops & Memory Layer):** The architecture features an asynchronous **Autonomous Event Loop**, isolated from the UI, continuously polling for `tool_use` or `<|eom_id|>` signals⁴. It utilizes **Tiered Memory**, storing transient tool outputs in the prompt context while persisting critical state observations in an integrated Vector Database for long-term retrieval.
4. **The Governance Abstraction (Security & Compliance Layer):** Finally, the MCP incorporates a non-negotiable **Security Middleware**. This layer enforces ephemeral sandboxing for code execution, intercepts outputs via prompt-injection classifiers, strips out unmapped PHI to maintain HIPAA compliance, and streams immutable state logs to an observability platform to satisfy SOC 2 and GDPR audit requirements¹⁴.

By bridging dynamic, reasoning-capable foundation models with heavily governed, isolated execution environments, this template provides the foundational blueprint for secure, highly capable enterprise agentic systems.

Works cited

1. Create a Message - Claude API Reference, <https://platform.claude.com/docs/en/api/messages/create>
2. Prompting best practices - Claude Platform Docs, <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
3. Tool-Calling with Llama 3.1: Integrating Real-Time Data in Open-Source Models, <https://www.analyticsvidhya.com/blog/2024/07/tool-calling-in-llama-3-1/>
4. Llama 3.1 | Model Cards and Prompt formats - Meta for Developers, https://developer.meta.com/ai/docs/model-cards-and-prompt-formats/llama3_1/
5. zac/phi4-tools/template - Ollama, <https://ollama.com/zac/phi4-tools/blobs/5620746093d7>
6. microsoft/Phi-4-reasoning · phi-4-reasoning chat template forces system prompt?, <https://huggingface.co/microsoft/Phi-4-reasoning/discussions/6>
7. Qwen2.5: A Party of Foundation Models, <https://qwen.ai/blog?id=qwen2.5>
8. Dataset format/ prompt template for fine tuning Qwen 2.5-Coder Instruct - Stack Overflow, <https://stackoverflow.com/questions/79122056/dataset-format-prompt-template-for-fine-tuning-qwen-2-5-coder-instruct>
9. <https://ai.google.dev/gemma/docs/functiongemma/formatting-and-best-practices>
10. Effective context engineering for AI agents - Anthropic, <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
11. GitHub - Piebald-AI/claude-code-system-prompts: All parts of Claude Code's system prompt, 27 builtin tool descriptions, sub agent prompts (Plan/Explore/Task), utility prompts (CLAUDE.md, compact, statusline, magic docs, WebFetch, Bash cmd, security review, agent creation). Updated for each Claude Code version., <https://github.com/Piebald-AI/claude-code-system-prompts>
12. Tool calls in LLaMa 3.1 - Braintrust, https://www.braintrust.dev/docs/cookbook/recipes/LLaMa-3_1-Tools
13. The Claude Developer Guide in Python — Tools - GoPenAI, <https://blog.gopenai.com/the-claude-developer-guide-in-python-tools-dbecb10469e9>
14. Computer use tool - Claude Platform Docs, <https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool>
15. What is the exact system prompt that gets inserted when calling a tool? - API, <https://community.openai.com/t/what-is-the-exact-system-prompt-that-gets-inserted-when-calling-a-tool/1075113>
16. microsoft / phi-4-multimodal-instruct - NVIDIA API Documentation, <https://docs.api.nvidia.com/nim/reference/microsoft-phi-4-multimodal-instruct>
17. PhiCookBook/md/02.Application/07.FunctionCalling/Phi4/FunctionCallingBasic/RE

- ADME ... - GitHub,
<https://github.com/microsoft/PhiCookBook/blob/main/md/02.Application/07.FunctionCalling/Phi4/FunctionCallingBasic/README.md>
18. Guides: Get started with Computer Use - AI SDK,
<https://ai-sdk.dev/cookbook/guides/computer-use>
 19. Claude 3.7 Sonnet and Claude Code - Anthropic,
<https://www.anthropic.com/news/claude-3-7-sonnet>
 20. microsoft/Phi-4-mini-instruct - Hugging Face,
<https://huggingface.co/microsoft/Phi-4-mini-instruct>
 21. Pricing - Claude Platform Docs,
<https://platform.claude.com/docs/en/about-claude/pricing>
 22. Mid-conversation system messages and tool changes - Claude Platform Docs,
<https://platform.claude.com/docs/en/build-with-claude/mid-conversation-system-messages>
 23. Optimizing Claude 3.7 Sonnet API Costs: A Guide to Managing Anthropic's Extended Thinking Budgets | Morfless,
<https://morfless.com/blog/claude-3-7-sonnet-thinking-budgets-cost-optimization>
 24. Extended thinking - Claude Platform Docs,
<https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
 25. Thinking / Reasoning | Docker Docs,
<https://docs.docker.com/ai/docker-agent/guides/thinking/>
 26. Practical Guide to Extended Thinking: How to Use Thinking Budgets in the Opus 4.7 Era,
<https://claudeapi.com/en/blog/dev-guides/claude-extended-thinking-practical-guide-2026/>
 27. generate_content - Google Gen AI Python SDK,
<https://googleapis-python-genai-70.mintlify.app/api/models/generate-content>
 28. Gemini thinking - Interactions API | Google AI for Developers,
<https://ai.google.dev/gemini-api/docs/thinking>
 29. How to include thinking process in output for deepseek-r1 : r/SillyTavernAI - Reddit,
https://www.reddit.com/r/SillyTavernAI/comments/1idnml3/how_to_include_thinking_process_in_output_for/
 30. anthropics/claude-agent-sdk-python - GitHub,
<https://github.com/anthropics/claude-agent-sdk-python>
 31. Claude Agent SDK in Python: First Agent to Workflows | Augment Code,
<https://www.augmentcode.com/guides/claude-agent-sdk-python>
 32. llama-cookbook/end-to-end-use-cases/agents/Agents_Tutorial/Tool_Calling_101.ipynb at main - GitHub,
https://github.com/meta-llama/llama-cookbook/blob/main/end-to-end-use-cases/agents/Agents_Tutorial/Tool_Calling_101.ipynb
 33. Tool Calling in Llama 3: A Step-by-step Guide To Build Agents | Composio,
<https://composio.dev/content/tool-calling-in-llama-3-a-guide-to-build-agents>
 34. Prompting Guide for Code Llama,

<https://www.promptingguide.ai/models/code-llama>

35. google genai sdk integration · langfuse · Discussion #4904 - GitHub,
<https://github.com/orgs/langfuse/discussions/4904>

36. Steering thinking - Claude Platform Docs,
<https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost>